

Write-Through

🌐 systemdesignschool.io/fundamentals/caching-write-patterns

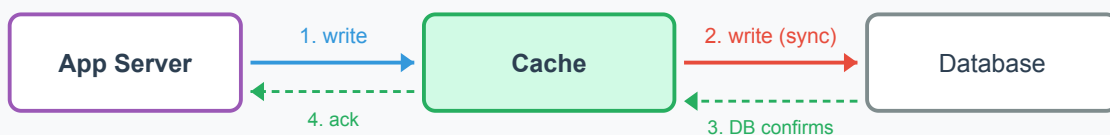


Write Patterns: Mutating Data

The previous articles covered how data enters the cache on read. But reads only solve half the problem. When the application writes new data or updates existing records, the cache and database can diverge. The write pattern determines how—and when—those two stores synchronize.

In **write-through**, every write goes to both the cache and the database as a single synchronous operation. The application sends the write to the cache layer. The cache stores the value and immediately forwards it to the database. Only after the database confirms does the cache acknowledge success to the application.

Write-Through: Synchronous Write to Both



Total latency = cache write + DB write (synchronous)



This keeps cache and database synchronized on every successful write. Subsequent reads typically find fresh data in the cache while the entry remains cached—though eviction, TTL, or node failures can still cause misses.

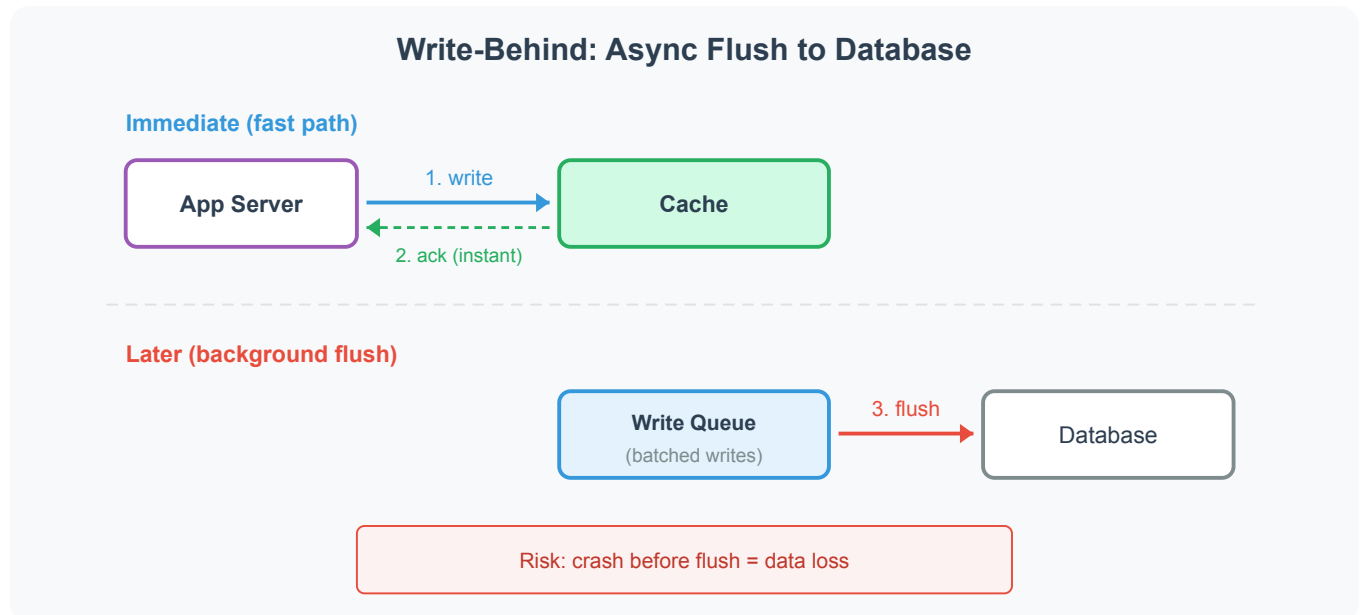
The cost is write latency. Every write now pays two round trips: one to the cache, one to the database. The extra hop adds a millisecond or more to each write. For read-heavy workloads with occasional writes, this tradeoff is acceptable. For write-heavy workloads, the added latency compounds.

There is also a cache pollution problem. Write-through stores every written value in cache, regardless of whether anyone reads it. An analytics pipeline writing thousands of events per second fills the cache with entries that no read request ever touches. Memory fills with write-once data while frequently-read entries get evicted.

Amazon DAX. [DynamoDB Accelerator](#) implements write-through transparently. The application uses a compatible API—DAX performs write-through, updating the cache and forwarding to DynamoDB. Reads hit DAX first.

Write-Behind (Write-Back)

In **write-behind**, the cache acknowledges writes immediately and flushes to the database later. The application writes to the cache. The cache acknowledges the write quickly. A background process batches cached writes and persists them to the database on a schedule or when a threshold is reached.



Write latency drops dramatically. The application waits only for the cache acknowledgment (typically much faster than a database round trip). Under heavy write bursts, the cache absorbs the spike and smooths it into steady database writes.

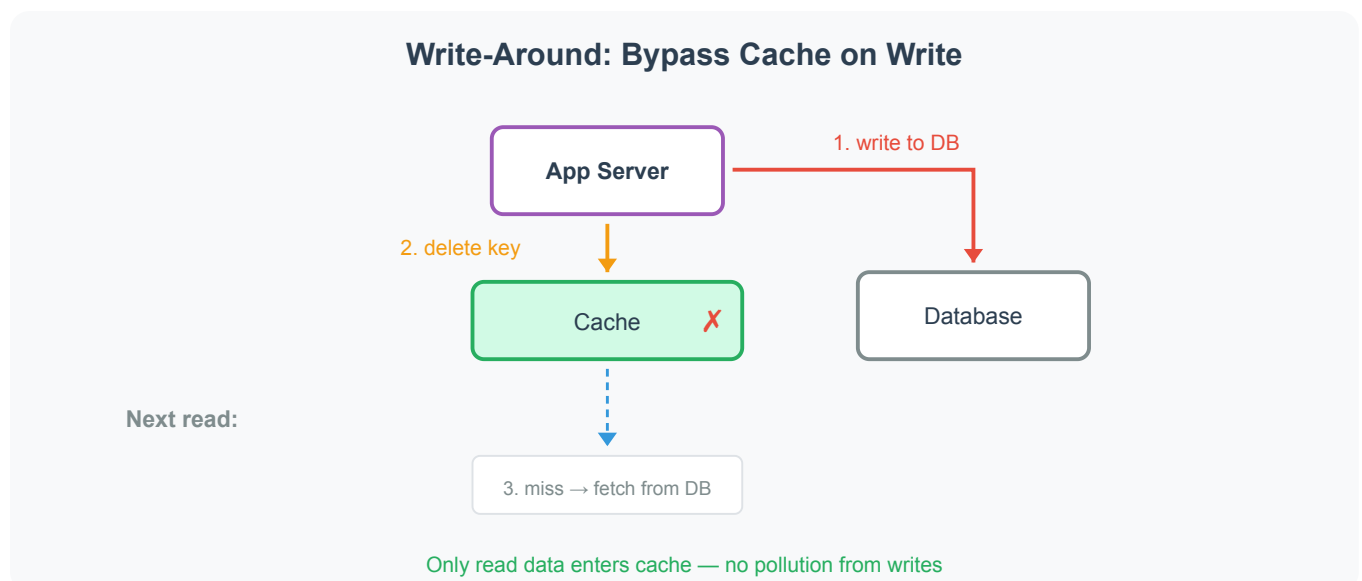
Batching provides a second benefit: **write coalescing**. For overwrite-style updates where intermediate states don't matter, if the same key updates 10 times in one second, only the final value needs to reach the database. Instead of 10 database writes, the flush sends 1. This reduces database load significantly for frequently-updated counters, session data, or user presence indicators.

The risk is data loss. In the simplest form, between the cache acknowledging the write and the background flush completing, data exists only in memory. If the cache node crashes during this window, unflushed writes are lost. Replication or durable write-ahead logging can mitigate this at the cost of added latency.

Write-behind fits workloads where speed matters more than durability: session stores, view counters, real-time analytics, recommendation scores. It does not fit financial transactions, order processing, or any data where loss is unacceptable.

Write-Around

In **write-around**, writes go directly to the database, bypassing the cache entirely. The application invalidates (deletes) the cache entry for the affected key. The next read triggers a cache miss, fetches fresh data from the database, and repopulates the cache.



This is the write-side partner of cache-aside. The application manages the cache explicitly: invalidate on write, populate on read. No write ever enters the cache directly.

Write-around avoids the cache pollution problem. Only data that is actually read enters the cache. A batch import of 100,000 records does not touch cache memory. Only the records that users subsequently request get cached.

The tradeoff is a cache miss on the next read after every write (assuming the invalidation succeeds). The user who updates their profile will experience one slow read (cache miss + database fetch) on the next page load. For most applications, this single-miss penalty is

negligible compared to the memory savings.

Choosing a Write Pattern

The choice depends on your consistency requirements and write volume.

Consideration	Write-Through	Write-Behind	Write-Around
Consistency	Strong (synchronous)	Eventual (async flush)	Strong (invalidate + refetch)
Write latency	Higher (cache + DB)	Lowest (cache only)	Normal (DB only)
Data loss risk	None	Yes (crash before flush)	None
Cache pollution	Yes (all writes cached)	Yes (all writes cached)	No (only reads populate)
Best for	Read-heavy, consistency-critical	Write-heavy, loss-tolerant	Mixed workloads, large writes

Many production systems use **write-around** (cache-aside for writes). It keeps the cache lean, avoids data loss risk, and pairs naturally with the cache-aside read pattern from the previous article. Write-through adds value when reads of recently-written data are frequent and consistency is critical. Write-behind is reserved for high-throughput scenarios where the durability tradeoff is explicitly accepted.

When these patterns fail, the consistency problems that arise from writing to two systems (cache and database) become the focus.